`

Project Report

Maze Transversal Game

Name :Rana Muhammad Ahmed

Enrollment :01-134241-039

Subject :OOP(lab)

Section :Bs(Cs)2A

Instructor :Muhammad Siddique

Bahria university shangrilla road ,Sector E-8 ,Islamabad

`

Table of Contents

Project Report : Maze Transversal Game **3**

1. Problem Statement 3

2. Objectives **3**

3. Source Code 5

Libraries Included 5

Header Files Included 5

Header Files Code 5

Cell.h : 5

Maze.h: 6

Game.h: 9

Player.h: 11

ProfessionalPlayer.h: 12

AmateurPlayer.h: 12

4. Sample Outputs 12

Main Code: 12

Output: 13

Main Code: 13

Output: 13

5.Conclusion 15

`

Project Report : Maze Transversal Game

1. Problem Statement

The goal of this project is to develop a maze traversal game that allows players to navigate through a maze, solving various challenges to reach the exit. The game will involve dynamic maze generation, player movement within the maze, and the calculation of a score based on the time taken to solve the maze. The project will include features such as saving player details, including their name, age, scores, and the number of mazes solved, to provide a personalized experience. This will also involve managing player records through file handling techniques.

The game will be built using object-oriented programming (OOP) principles, focusing on the creation of classes to represent different entities, such as the Player, Maze, and Game objects. The use of inheritance, polymorphism, and exception handling will also be incorporated to ensure the game is flexible and robust. Through this project, the player will experience a challenging, interactive game that encourages problem-solving and strategic thinking, while also demonstrating the application of OOP concepts in game development. The goal is to create an engaging, functional, and extensible game that could be enhanced with additional features in the future.

2. Objectives

1. To design and implement a maze traversal game that allows players to navigate through dynamically generated mazes.
2. To calculate and display the player's score based on the time taken to solve the maze.
3. To create suitable modular class hierarchy that can be modified and has all the necessary game features and components
4. To use separate compilation principle.
5. To implement file handling for saving and retrieving player records and scores.
6. To incorporate inheritance and polymorphism in the game design for better code reuse and flexibility.
7. To use exception handling to manage potential errors and ensure the game runs smoothly.
8. To provide an engaging and interactive experience for the player while demonstrating core object-oriented programming principles.
9. To allow for the future expansion of the game with additional features, such as different maze types or advanced gameplay mechanics.

`

2. UML Diagram

This is the UML diagram for the game showing classes and their relationships

`

3. Source Code

Libraries Included

<SFML/Graphics.hpp>: Provides functions and classes for creating and manipulating graphical elements, such as windows, shapes, and textures, used for rendering the game visuals.

<SFML/Audio.hpp>: Allows handling and playing audio files, useful for adding sound effects or background music to the game.

<iostream>: Provides functions for basic input and output operations, such as reading from the console (cin), printing to the console (cout), and using manipulators like endl.

<fstream>: Facilitates file handling operations, such as reading from and writing to files, enabling the storage and retrieval of player data or game states.

<string>: Used for handling and manipulating strings, allowing operations like concatenation, comparison, and string conversion.

<stack>: Provides a stack data structure, which can be used to manage data in a last-in, first-out (LIFO) order

<r**andom>**: Offers random number generation functions, useful for generating unpredictable values, such as random maze generation or player movements.

Header Files Included

"Cell.h": containing the **cell** class.

"Maze.h": containing the **Maze** class.

"Game.h": containing the **game** class.

"Player.h": containing the **player** class.

"ProfessionalPlayer.h": containing the **ProfessionalPlayer** class.

"AmateurPlayer.h": containing the **AmateurPlayer** class.

Header Files Code

Cell.h :

```
#pragma once
#include <SFML/Graphics.hpp>
using namespace sf;
class Cell {
    int x, y;
    int pos;
    float thickness;
```

`

```
    bool walls[4] = { true, true, true, true };
    bool isVisited = false;
    bool isActive = false;
    float cellSize;
public:
    friend class Game;
    friend class Maze;
    Cell(int posX = 0, int posY = 0, int posIndx = 0) : pos(posIndx), x(posX),
y(posY), cellSize(30.f), thickness(2.f) {}
    void drawCell(RenderWindow& window) {
        RectangleShape rect;
        if (isActive) {
            rect.setFillColor(Color(255, 245, 0));
            rect.setSize(Vector2f(cellSize, cellSize));
            rect.setPosition(x, y);
            window.draw(rect);
        }
        rect.setFillColor(Color());
        //top wall
        if (walls[0]) {
            rect.setSize(Vector2f(cellSize, thickness));
            rect.setPosition(x, y);
            window.draw(rect);
        }
        //right wall
        if (walls[1]) {
            rect.setSize(Vector2f(thickness, cellSize));
            rect.setPosition(x + cellSize, y);
            window.draw(rect);
        }
        //bottom wall
        if (walls[2]) {
            rect.setSize(Vector2f(cellSize + thickness, thickness));
            rect.setPosition(x, y + cellSize);
            window.draw(rect);
        }
        //left wall
        if (walls[3]) {
            rect.setSize(Vector2f(thickness, cellSize));
            rect.setPosition(x, y);
            window.draw(rect);
        }
    }
};
```

Maze.h:

```
#pragma once
#include <SFML/Graphics.hpp>
#include "Cell.h"
#include<stack>
#include <random>
using namespace std;
using namespace sf;
class Maze {
    int Mazesize;
```

```
    Cell* grid;
    int cellWidth;
public:
    friend class Game;
    Maze(int size = 30, int cellWidth = 30) : Mazesize(size), grid(new Cell[Mazesize
* Mazesize]), cellWidth(cellWidth) {
        for (int i = 0, k = 0; i < Mazesize; i++) {
            for (int j = 0; j < Mazesize; j++, k++) {
                grid[k] = Cell(j * cellWidth, i * cellWidth, k);
            }
        }
    }
    void resetMaze() {
        for (int i = 0; i < Mazesize * Mazesize; i++) {
            for (int j = 0; j < 4; j++) {
                grid[i].walls[j] = true;
                grid[i].isVisited = false;
                grid[i].isActive = false;
            }
        }
    }
    void removeWallsBetween(Cell& current, Cell& chosen) {
        if (current.pos - Mazesize == chosen.pos) {
            current.walls[0] = false;
            chosen.walls[2] = false;
        }
        else if (current.pos + 1 == chosen.pos) {
            current.walls[1] = false;
            chosen.walls[3] = false;
        }
        else if (current.pos + Mazesize == chosen.pos) {
            current.walls[2] = false;
            chosen.walls[0] = false;
        }
        else if (current.pos - 1 == chosen.pos) {
            current.walls[3] = false;
            chosen.walls[1] = false;
        }
    }
    void makeMaze() {
        resetMaze();
        stack<Cell> stack;
        grid[0].isVisited = true;
        stack.push(grid[0]);
        while (!stack.empty()) {
            Cell current = stack.top();
            stack.pop();
            int pos = current.pos;
            int* neighbors = new int[4];
            int neighborCount = 0;

            if (pos % Mazesize != 0 && pos > 0) {
                Cell& left = grid[pos - 1];
                if (!left.isVisited) {
                    neighbors[neighborCount++] = pos - 1;
                }
            }
            if ((pos + 1) % (Mazesize) != 0 && pos < Mazesize * Mazesize) {
```

```
                Cell& right = grid[pos + 1];
                if (!right.isVisited) {
                    neighbors[neighborCount++] = pos + 1;
                }
            }
            if ((pos + Mazesize) < Mazesize * Mazesize) {
                Cell& bottom = grid[pos + Mazesize];
                if (!bottom.isVisited) {
                    neighbors[neighborCount++] = pos + Mazesize;
                }
            }
            if ((pos - Mazesize) > 0) {
                Cell& top = grid[pos - Mazesize];
                if (!top.isVisited) {
                    neighbors[neighborCount++] = pos - Mazesize;
                }
            }

            if (neighborCount > 0) {
                random_device dev;
                mt19937 rng(dev());
                uniform_int_distribution<mt19937::result_type> dist6(0,
neighborCount - 1);
                int randNeighborPos = dist6(rng);
                Cell& chosen = grid[neighbors[randNeighborPos]];
                stack.push(current);
                removeWallsBetween(grid[current.pos], chosen);
                chosen.isVisited = true;
                stack.push(chosen);
            }

            delete[] neighbors;
        }
    }
    void handleMove(Event event, int& currentPos) {
        if (event.key.code == Keyboard::Left || event.key.code == Keyboard::A) {
            if (!grid[currentPos].walls[3] && !grid[currentPos - 1].walls[1]) {
                currentPos = currentPos - 1;
                grid[currentPos].isActive = true;
            }
        }
        else if (event.key.code == Keyboard::Right || event.key.code == Keyboard::D)
{
            if (!grid[currentPos].walls[1] && !grid[currentPos + 1].walls[3]) {
                currentPos = currentPos + 1;
                grid[currentPos].isActive = true;
            }
        }
        else if (event.key.code == Keyboard::Up || event.key.code == Keyboard::W) {
            if ((currentPos - Mazesize) < 0) {
                return;
            }
            if (!grid[currentPos].walls[0] && !grid[currentPos - Mazesize].walls[2])
{
                currentPos = currentPos - Mazesize;
                grid[currentPos].isActive = true;
            }
        }
```

```
`


        else if (event.key.code == Keyboard::Down || event.key.code == Keyboard::S)
{
            if ((currentPos + Mazesize) > Mazesize * Mazesize) {
                return;
            }
            if (!grid[currentPos].walls[2] && !grid[currentPos + Mazesize].walls[0])
{
                currentPos = currentPos + Mazesize;
                grid[currentPos].isActive = true;
            }
        }
    }
    ~Maze() {
        delete[] grid;
    }
};
```

Game.h:

```
#pragma once
#include <SFML/Graphics.hpp>
#include<SFML/Audio.hpp>
#include "Maze.h"
#include "ProfessionalPlayer.h"
#include "AmateurPlayer.h"
using namespace sf;
class Game {
    int currentPos;
    Maze maze;
    Player* player;
public:
    friend istream& operator>>(istream& is, Game& game);
    friend ostream& operator<<(ostream& os, Game& game);
    Game(int mazesize = 30, int pos = 0, string name = "unknown", int age = 0)
        : currentPos(pos), maze(mazesize) {
        char choice;
        cout << endl<< "Are you are proffesional player (y/n)" ;
        cin >> choice;
        if (choice=='y'||choice=='Y') {
            player = new ProfessionalPlayer(name, age);
        }
        else {
            player = new AmateurPlayer(name, age);
        }
    }
    void playGame() {
        SoundBuffer BgBuffer;
        if (!BgBuffer.loadFromFile("mazebg.wav")) {
            cout << "could not load bg music";
        }
        Sound BgSound;
        BgSound.setBuffer(BgBuffer);
        BgSound.play();
        BgSound.setLoop(true);
        player->setPlayer();
        RenderWindow window(VideoMode((maze.Mazesize * maze.cellWidth),
(maze.Mazesize * maze.cellWidth)), "Maze Game");
        window.setFramerateLimit(30);
        window.setVerticalSyncEnabled(true);
```

`

```
        maze.makeMaze();
        maze.grid[currentPos].isActive = true;
        RectangleShape currentPosRect;
        currentPosRect.setFillColor(Color(52, 148, 132));
        currentPosRect.setSize(Vector2f(maze.cellWidth, maze.cellWidth));
        RectangleShape finishRect;
        finishRect.setFillColor(Color(233, 82, 82));
        finishRect.setSize(Vector2f(maze.cellWidth, maze.cellWidth));
        while (window.isOpen()) {
            Event event;
            while (window.pollEvent(event)) {
                switch (event.type) {
                case Event::Closed:
                    window.close();
                    break;
                case Event::KeyPressed:
                    maze.handleMove(event, currentPos);
                    break;
                default:
                    break;
                }
            }
            if (currentPos == (maze.Mazesize * maze.Mazesize - 1)) {
                maze.makeMaze();
                currentPos = 0;
                maze.grid[currentPos].isActive = true;
                player->incrementGamesPlayed();
            }
            window.clear(Color(190, 255, 244));
            for (int i = 0; i < (maze.Mazesize * maze.Mazesize); i++) {
                maze.grid[i].drawCell(window);
            }
            currentPosRect.setPosition(maze.grid[currentPos].x,
maze.grid[currentPos].y);
            window.draw(currentPosRect);
            finishRect.setPosition(maze.grid[maze.Mazesize * maze.Mazesize - 1].x,
maze.grid[maze.Mazesize * maze.Mazesize - 1].y);
            window.draw(finishRect);
            window.display();
        }
        player->getPlayer();
        cout << "Player rating score: " << player->ratingScore() << endl;
        player->writeToFile();
    }
    ~Game() {
        delete player;
    }
};
ostream& operator<<(ostream& os, Game& game) {
    game.playGame();
    return os;
}
istream& operator>>(istream& is, Game& game) {
    cout << endl<< "Update player details";
    game.player->setPlayer();
    return is;
}
```

`

Player.h:

```
#pragma once
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
class Player {
protected:
    string name;
    int age;
    int gamesPlayed;
public:
    Player(string name = "unknown", int age = 0) : name(name), age(age),
gamesPlayed(0) {}
    virtual ~Player() {}
    virtual int ratingScore() const = 0;
    void validationError() {
        cout << "Invalid input.Try again :";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    };
    void setPlayer() {
            cout << "Enter player name: ";
            cin.ignore();
            getline(cin, name);
            cout << "Enter player age: ";
            while (!(cin >> age)&&(age<0))
            {
                validationError();
            };
    }
    void getPlayer() const {
        cout << "Player name: " << name << endl;
        cout << "Player age: " << age << endl;
        cout << "Player games played: " << gamesPlayed << endl;
    }
    void incrementGamesPlayed() {
        gamesPlayed++;
    }
    void writeToFile() const {
        ofstream file;
        try {
            file.open("player.txt", ios::app);
            if (!file) {
                throw runtime_error("Unable to open file");
            }
            file << name << " " << age << " " << gamesPlayed << " " << ratingScore()
<< endl;
            file.close();
        }
        catch (const exception& e) {
            cerr << "An error occurred while writing to file: " << e.what() << endl;
        }
    }
};
```

`

ProfessionalPlayer.h:

```
#pragma once
#include "Player.h"
class ProfessionalPlayer : public Player {
public:
    ProfessionalPlayer(string name = "unknown", int age = 1) : Player(name, age) {}
    int ratingScore() const override {
        if (age <= 0) {
            return 0;
        }
        return (gamesPlayed * 20) / age;
    }
};
```

AmateurPlayer.h:

```
#pragma once
#include "Player.h"
class AmateurPlayer : public Player {
    AmateurPlayer(string name = "unknown", int age = 1) : Player(name, age) {}
    int ratingScore() const override {
        if (age <= 0) {
            return 0;
        }
        return gamesPlayed * 10;
    }
};
```

---

4. Sample Outputs

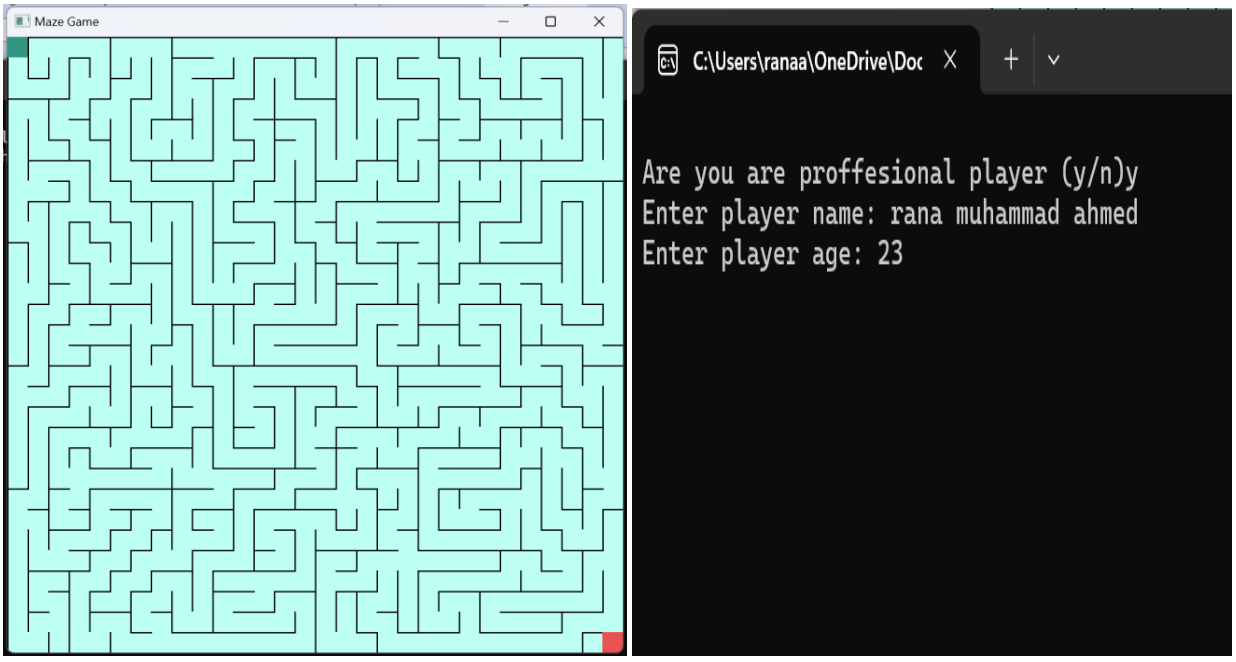
Basic Game UI:

Main Code:

```
#include"Game.h"
#include <iostream>
int main() {
      Game game;
      game.playGame();
      return 0;
}
```

`

Output:

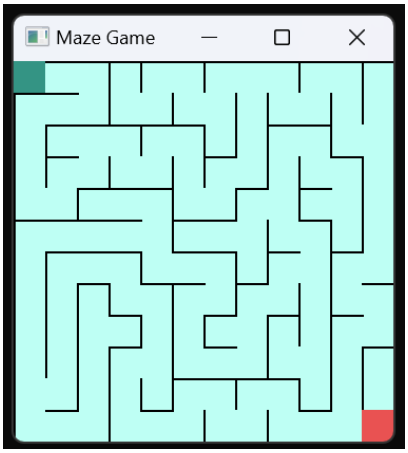


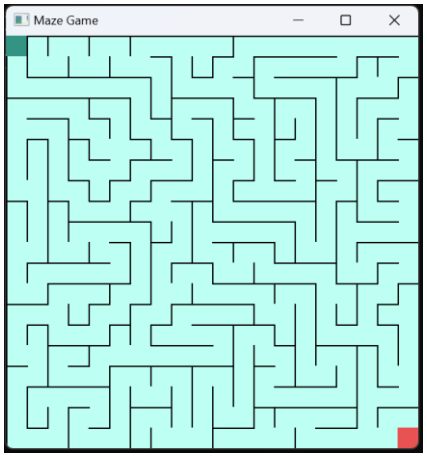
Dynamic maze :

Main Code:

```
#include"Game.h"
#include <iostream>
int main() {
	Game g1(12),g2(20);
	g1.playGame();
	g2.playGame();
	return 0;
}
```

Output:





Maze of two different sizes are generated by parameterized construction

`

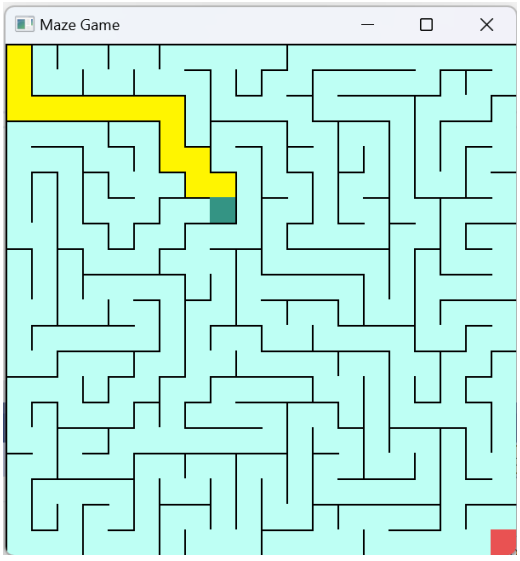
File saving mechanism :

player.txt

```
1   name age gamesplayed rattingscore
2   rananew 21 4 40
3   rana 21 2 1
4   rana 21 0 0
5   rana 21 1 0
6   rana 21 1 0
7   rana 23 1 0
8   Rana 0 1 0
9   Rana 0 0 0
10  rana muhammad ahmed 0 0 0
11  rana muhammad ahmed 0 0 0
12  rana ahmed 0 0 0
13  Rana muhammad Ahmed 30 0 0
14  rana muhammad ahmed 23 0 0
15  rana 11 0 0
16
```

Saving the player record after he closes the game window

Keeping Track of movements :



Highlighting the players path by yellow color

`

5.Conclusion

In conclusion, this project demonstrates the successful development of a Maze Traversal Game using the SFML library in C++. The project integrates essential programming concepts such as inheritance, polymorphism, file handling, Operator overloading, Composition and exception handling, highlighting a strong foundation in Object-Oriented Programming (OOP). By combining graphical rendering, sound effects, and interactive gameplay mechanics, the game provides an engaging user experience.

The game efficiently utilizes SFML for creating visual elements and managing audio, while standard C++ libraries such as iostream, fstream, and string handle input/output operations and file management. Additionally, the use of stack and random demonstrates advanced problem-solving techniques, such as implementing maze generation and traversal logic.

This project emphasizes the importance of modular code, reusable classes, and error handling to ensure reliability and maintainability. It also incorporates player record management through file handling, providing a personalized experience for users.

Overall, the Maze Traversal Game not only meets the academic requirements of demonstrating OOP principles but also serves as a creative and interactive application that can be expanded in the future with features like dynamic mazes, player leaderboards, and multiplayer modes. This project effectively bridges theoretical knowledge with practical application, highlighting the versatility of C++ in game development.